PRISTINI
SCHOOL OF AI

Databases Engineering



Dr. **Kalthoum Zaouali**

kalthoum.zaouali@pristini-international.tn

Databases Engineering Chapters



1. Overview of SQL Databases
2. Database Structures and Management
3. Advanced SQL queries for data analysis

PRISTINI SCHOOL OF AI

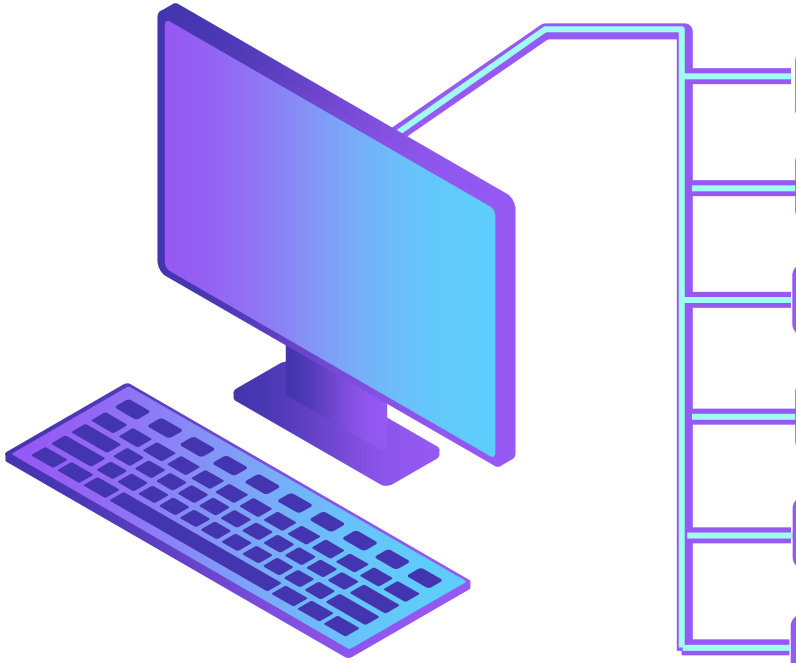
Database Structures and Management

Chapter II



©Dr. Kalthoum Zaouali 2026-2027

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

©Dr. Kalthoum Zaouali 2026-2027

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

©Dr. Kalthoum Zaouali 2026-2027

Filtering data using AND, OR, NOT, IN, BETWEEN, LIKE logical operators

- The **WHERE clause** in **SQL SELECT, UPDATE and DELETE** statements is used to filter data.
- It's possible to use **logical operators** and some other operators when formulating **filter conditions**.

Filtering data using AND operator

- The **WHERE clause** can contain one or many **AND operators**.
- The **AND operator** is used to filter records based on more than one condition.
- **Syntax :**

```
SELECT <column1>, <column2>, <...>
FROM <table_name>
WHERE <condition1> AND <condition2> AND <condition3>, <...>;
```

Filtering data using AND operator: Example

❖ Select all customers from Mexico that starts with the letter 'A':

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

➢ **Syntax:**

```
SELECT *
FROM Customers
WHERE Country = 'Mexico' AND CustomerName LIKE 'A%';
```

Filtering data using OR operator

- The **WHERE clause** can contain one or more OR operators.
- The **OR operator** is used to filter records based on more than one condition.
- **Syntax :**

```
SELECT <column1>, <column2>, <...>
FROM <table_name>
WHERE <condition1> OR <condition2> OR <condition3>, <...>;
```

Filtering data using OR operator: Example

❖ Select all customers from Germany or Mexico:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

➢ **Syntax:**

```
SELECT *
FROM Customers
WHERE Country = 'Germany' OR Country = 'Mexico';
```

Filtering data using NOT operator

- The **NOT** operator is used in combination with other operators to give the opposite result, also called the negative result.

- **Syntax :**

```
SELECT <column1>, <column2>, <...>
FROM <table_name>
WHERE NOT <condition1>
```

Filtering data using NOT operator: Example

❖ Select only the customers that are NOT from Mexico:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

➢ **Syntax:**

```
SELECT *
FROM Customers
WHERE NOT Country = 'Mexico';
```

Filtering data using IN operator

- The **IN operator** allows you to specify multiple values in a **WHERE clause**.
- The **IN operator** is a shorthand for multiple **OR** conditions.

- **Syntax :**

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <column_name> IN ( <value1>, <value2>, … )
```

Filtering data using IN operator: Example

❖ Return all customers from 'Germany', 'Sweden', or 'UK'

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

➢ **Syntax:**

```
SELECT *
FROM Customers
WHERE Country IN ('Germany', 'Sweden', 'UK');
```

Filtering data using BETWEEN operator

- The **BETWEEN operator** selects values within a given range. The values can be numbers, text, or dates.

- The **BETWEEN operator** is inclusive: begin and end values are included.
  - It can be used as an alternative to (**>=**) and (**<=**) comparison operators.

- **Syntax :**

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <column_name> BETWEEN <value1> AND <value2>
```

Filtering data using BETWEEN operator: Example

❖ Selects all products with a price between 10 and 20:

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

➢ **Syntax:**

```
SELECT *
FROM Products
WHERE Price BETWEEN 10 AND 20;
```

Filtering data using LIKE operator

- The **LIKE operator** is used in a **WHERE clause** to search for a specified pattern in a column.

- There are two wildcards often used in conjunction with the **LIKE operator**:
  - The percent sign '**%**' represents **zero, one, or multiple characters**
  - The underscore sign '**_**' represents **one, single character**

- **Syntax :**

```
SELECT <column1>, <column2>, <...>
FROM <table_name>
WHERE <column_name> LIKE <pattern>
```

Filtering data using LIKE operator: Example

❖ Select all customers that starts with the letter "a":

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

➢ **Syntax:**

```
SELECT *
FROM Customers
WHERE CustomerName LIKE 'a%';
```

Filtering data using LIKE operator: Example

- Let's take the example of a scenario where a pattern match must be performed on the EmployeeName column in the Employee's table to filter data.

ID	EmployeeName	Role	AnnualSalary
1	Mario Gollini	Manager	70000
2	Adrian Gollini	Assistant Manager	65000
3	Giorgos Dioudis	Head Chef	50000
4	Fatma Kaya	Assistant Chef	45000
5	Elena Salvai	Head Waiter	40000
6	John Millar	Receptionist	35000
7	Adam Collins	Assistant Manager	65000

Filtering data using LIKE operator: Example

- Examples of patterns based on the LIKE and the Employees' table:

Pattern	How it works
WHERE EmployeeName LIKE 'a%'	Finds EmployeeName values that start with "a". Matches with records that have IDs of 2 and 7.
WHERE EmployeeNameLIKE '%i'	Finds EmployeeName values that end with "i". Matches with the records that have IDs 1, 2, and 5.
WHERE EmployeeName LIKE '%li%'	Finds EmployeeName values that have "li" in any position. Matches with the record that have IDs 1, 2 and 7.
WHERE EmployeeName LIKE '_a%'	Finds EmployeeName values that have "a" in the second position. Matches with the records that have IDs 1 and 4.
WHERE EmployeeName LIKE 'a___%'	Finds EmployeeName values that start with "a" and are at least 4 characters in length. Matches with records that have IDs of 2 and 7.
WHERE EmployeeName LIKE 'a%i'	Finds EmployeeName values that start with "a" and end with "i". Matches with the record that has an ID of 2.

MySQL aliases

- SQL **aliases** are used to give a table, or a column in a table, **a temporary name**.
- Aliases are often used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the **AS** keyword.

✓ When alias is used on column:

Syntax:

```
SELECT <column_name> AS <alias_name>
FROM <table_name>;
```

✓ When alias is used on column:

Syntax:

```
SELECT <column_name(s)>
FROM <table_name> AS <alias_name>;
```

MySQL aliases

- Aliases can be useful when:
  - There are more than one table involved in a query
  - Functions are used in the query
  - Column names are big or not very readable
  - Two or more columns are combined together

MySQL aliases: Examples

❖ Alias for Columns:

- The following SQL statement creates two aliases, one for the CustomerID column and one for the CustomerName column:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Syntax:

```
SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers;
```

MySQL aliases: Examples

❖ Using Aliases With a Space Character

- If you want your alias to contain one or more spaces, like "My Great Products", surround your alias with square brackets or double quotes.

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

Syntax:

```
SELECT ProductName AS [My Great Products]
 FROM Products;
```

MySQL aliases: Examples

❖ Concatenate Columns:

❖ Create an alias named "Address" that combine four columns (Address, PostalCode, City and Country):

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Syntax:

```
SELECT CustomerName, Address + ', ' + PostalCode + ' '
+ City + ', ' + Country AS Address;
```

MySQL aliases: Examples

❖ Alias for Tables

- The following SQL statement selects all the orders from the customer with CustomerID=4 (Around the Horn). We use the "Customers" and "Orders" tables, and give them the table aliases of "c" and "o" respectively:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10383	4	8	1996-12-16	3

Syntax:

```
SELECT o.OrderID, o.OrderDate, c.CustomerName
FROM Customers AS c, Orders AS o
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```

©Dr. Kalthoum Zaouali 2026-2027

Joining tables

- A **JOIN** in a database links records of data between one or multiple tables based on a common column between them.

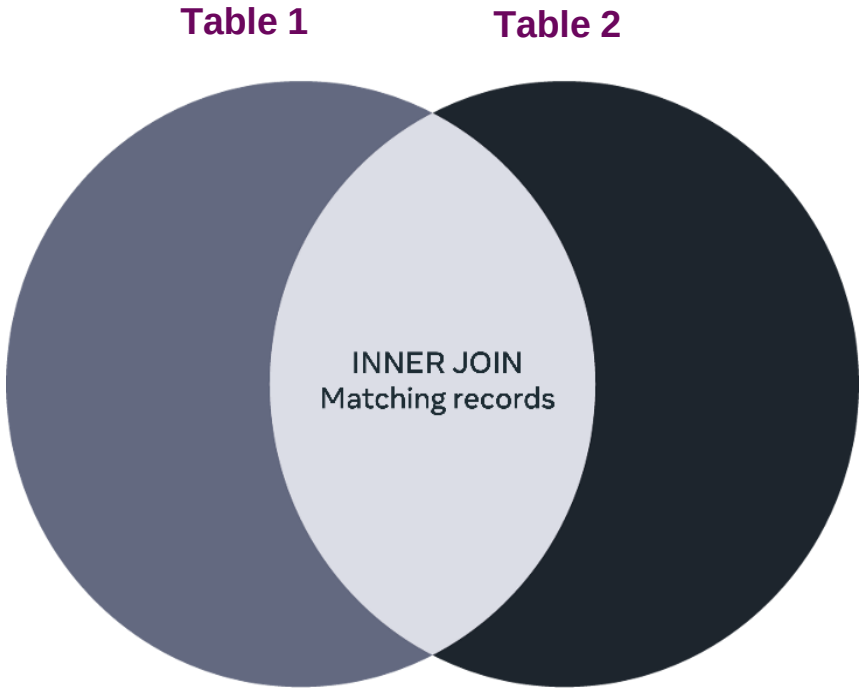
Why do you need to use a JOIN?

- *Sometimes you want to find information about a specific activity or object in the database, where the relevant information exists in more than one table.*
- You can use the SQL JOIN clause to query the required data from multiple tables.

- There are four different types of joins supported in MySQL :
  - **INNER JOIN**
  - **LEFT JOIN**
  - **RIGHT JOIN**
  - **SELF-JOIN**

Joining tables: INNER JOIN

- The **INNER JOIN** type returns records of data that have matching values in the joined tables.



Joining tables: INNER JOIN

❖ **Example:** The Little Lemon restaurant database includes Customers and Bookings tables:

- **Customers table:**

CustomerID	FullName	PhoneNumber
1	Vanessa McCarthy	757536378
2	Marcos Romero	757536379
3	Hiroki Yamane	757536376
4	Anna Iversen	757536375
5	Diana Pinto	757536374

- **Bookings table:**

BookingID	BookingDate	TableNumber	NumberOfGuests	CustomerID
10	2021-11-11	7	5	1
11	2021-11-10	5	2	2
12	2021-11-10	3	2	4

Joining tables: INNER JOIN

- Assume that you want to return the full name and booking ID of customers who made bookings.
- You can use the **INNER JOIN clause** to extract records of data from the Customers and the Bookings tables based on the matching customer ID value.

Syntax:

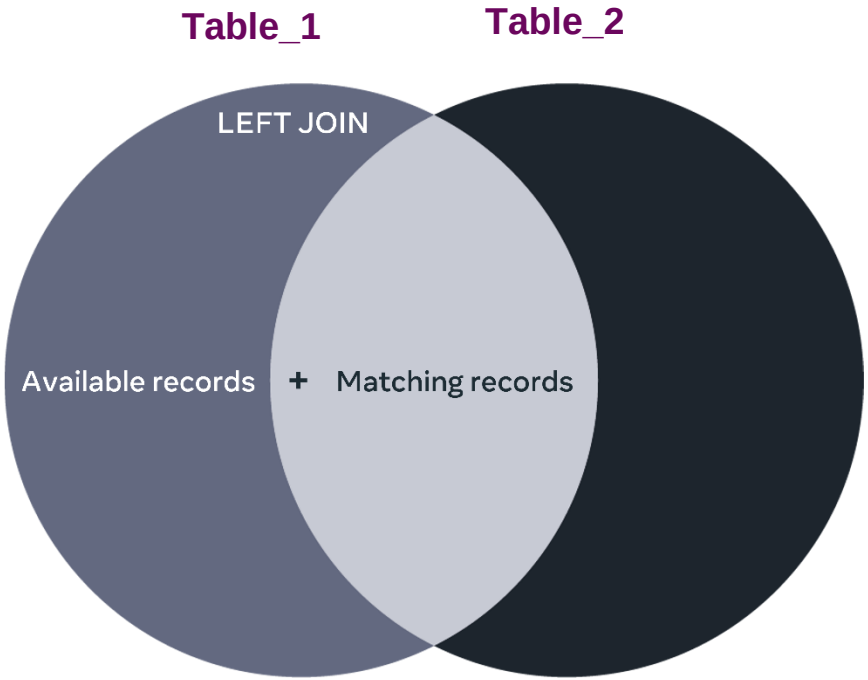
```
SELECT Customers.FullName, Bookings.BookingID
FROM Customers INNER JOIN Bookings
ON Customers.CustomerID = Bookings.CustomerID;
```

Result:

FullName	BookingID
Vanessa McCarthy	10
Marcos Romero	11
Anna Iversen	12

Joining tables: LEFT JOIN

- The **LEFT JOIN** returns all common records in a similar way to the INNER JOIN, plus all queried records from the left table regardless of whether there is a match in the right table or not.
- If there are no matching records in the right table, then **null** values will be inserted for the Table_2 IDs.



Joining tables: LEFT JOIN

- **Example:** You can use the **LEFT JOIN clause** to extract the full names and the booking IDs from the Customers and the Bookings tables.

- **Syntax:**

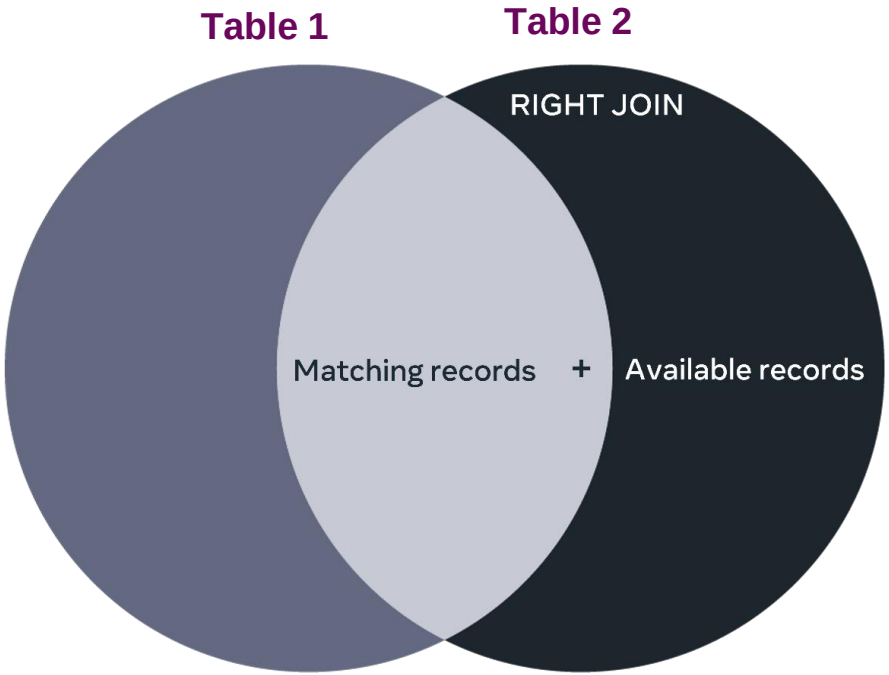
```
SELECT Customers.FullName, Bookings.BookingID
 FROM Customers LEFT JOIN Bookings
 ON Customers.CustomerID =  Bookings.CustomerID;
```

- **Result:**

FullName	BookingID
Vanessa McCarthy	10
Marcos Romero	11
Hiroki Yamane	NULL
Anna Iversen	12
Diana Pinto	NULL

Joining tables: RIGHT JOIN

- The **RIGHT JOIN** returns all common records in a similar way to the INNER JOIN, plus all queried records from the right table regardless of whether there is a match in the left table or not.
- If there are no matching records in the left table, then **null** values will be inserted for the Table_1 full names..



Joining tables: RIGHT JOIN

- **Example:** You can use the **RIGHT JOIN clause** to extract the full names and the booking IDs from the Customers and the Bookings tables.

- **Syntax:**

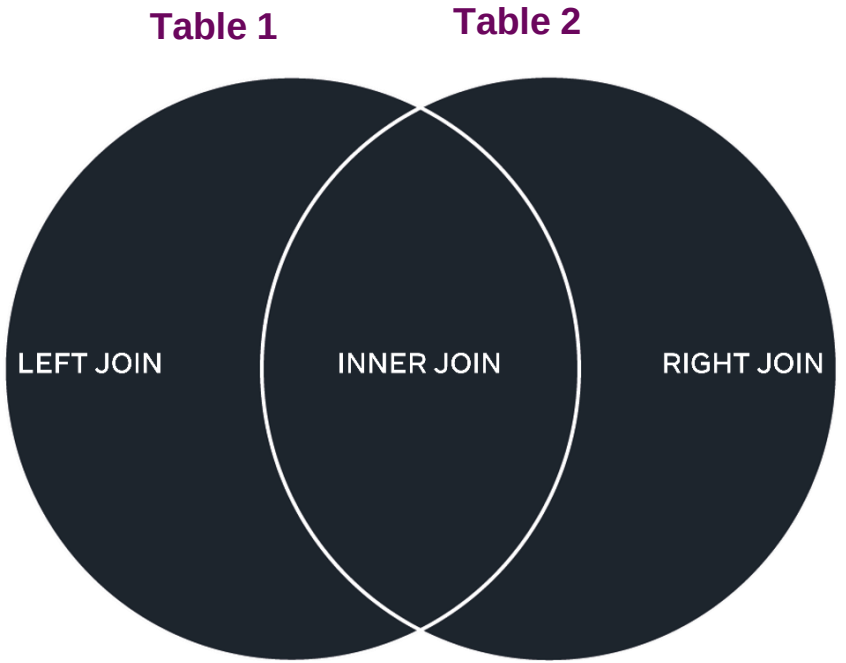
```
SELECT Customers.FullName, Bookings.BookingID
 FROM Customers RIGHT JOIN Bookings
 ON Customers.CustomerID = Bookings.CustomerID;
```

- **Result:**

FullName	BookingID
Vanessa McCarthy	10
Marcos Romero	11
Anna Iversen	12

Joining tables: FULL OUTER JOIN / FULL JOIN

- The **FULL OUTER JOIN** keyword returns all records when there is a match in left (Table1) or right (Table2) table records.
- **FULL OUTER JOIN** and **FULL JOIN** are the same.



- **Syntax:**

```
SELECT <column_name(s)>
FROM <Table1>
FULL OUTER JOIN <Table2>
ON <Table1.column_name> = <Table2.column_name>
WHERE <condition>;
```

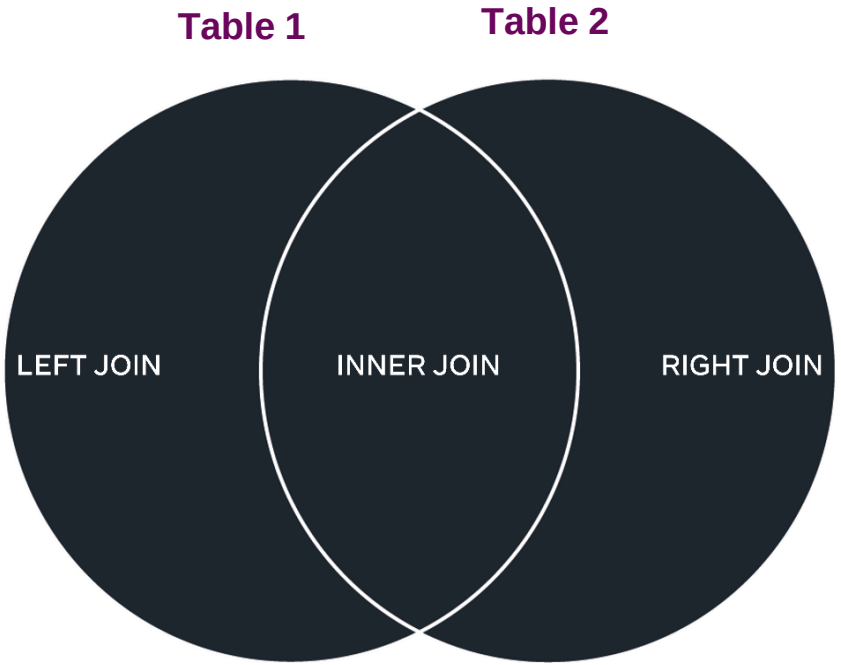
Joining tables: SELF JOIN

- The **SELF JOIN** is a special case where you need to join a table with itself to get specific information existing in the same table.

- You may choose the INNER JOIN, LEFT JOIN or RIGHT JOIN presented earlier to query the required data.

- **Syntax:**

```
SELECT column_name(s)
FROM Table1, Table2
WHERE condition;
```



Operators and clauses: UNION operator

- The **UNION** operator is used to combine the result-set of two or more SELECT statements.
  - Every SELECT statement within UNION must have the same number of columns
  - The columns must also have similar data types
  - The columns in every SELECT statement must also be in the same order

- **Syntax:**

```
SELECT <column_name(s)>
FROM <table_name(1)>
UNION
SELECT <column_name(s)>
FROM <table_name(2)>;
```

UNION operator: Examples

- Display the German cities (only distinct values) from both the "Customers" and the "Suppliers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico

SupplierID	SupplierName	ContactName	Address	City	PostalCode	Country	Phone
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	Londona	EC1 4SD	UK	(171) 555-2222
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans	70117	USA	(100) 555-4822

- **Syntax:**

```
SELECT City, Country FROM Customers
UNION ALL
SELECT City, Country FROM Suppliers
WHERE Country='Germany' ORDER BY City;
```

Operators and clauses: GROUP BY clause

- The **GROUP BY** clause in a SELECT statement is used to group rows in a table(s) based on a given column(s) into summary rows or subgroups.
  - If there is a WHERE clause in your SELECT statement, it should be placed after the WHERE clause.
  - After the GROUP BY keyword, place a list of comma-separated column names by which you want to group the data.
- The **GROUP BY** statement is often used with aggregate functions (**COUNT(), MAX(), MIN(), SUM(), AVG()**) to group the result-set by one or more columns.

- **Syntax:**

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <column_name><comparison operator>
GROUP BY <column_name(s)>
ORDER BY <column_name(s)>;
```

GROUP BY clause: Examples

- List the number of customers in each country:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

- **Syntax:**

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

GROUP BY clause: Examples

- List the number of customers in each country, sorted high to low:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

- **Syntax:**

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

GROUP BY clause with LEFT JOIN: Example

- List the number of orders sent by each shipper:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10248	90	5	1996-07-04	3
10249	81	6	1996-07-05	1
10250	34	4	1996-07-08	2

ShipperID	ShipperName	Phone
1	Speedy Express	(503) 555-9831
2	United Package	(503) 555-3199
3	Federal Shipping	(503) 555-9931

- **Syntax:**

```
SELECT Shippers.ShipperName, COUNT(Orders.OrderID) AS NumberOfOrders FROM Orders
LEFT JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
GROUP BY ShipperName;
```

Operators and clauses: HAVING clause

- If you want to filter your grouped data, use the **HAVING** clause.
  - The HAVING clause should appear after the GROUP BY clause.
  - In the HAVING clause, you can specify the filter condition(s) that needs to be applied to your grouped data.

- **Syntax:**

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <condition>
GROUP BY <column_name(s)>
HAVING <condition>
ORDER BY <column_name(s)>;
```

HAVING clause: Example

❖ List the number of customers in each country, sorted high to low (Only include countries with more than 4 customers):

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

❖ **Syntax:**

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 4
ORDER BY COUNT(CustomerID) DESC;
```

HAVING clause with GROUP BY and INNER JOIN : Example

- List the employees that have registered more than 5 orders:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10248	90	5	1996-07-04	3

EmployeeID	LastName	FirstName	BirthDate	Photo	Notes
1	Davolio	Nancy	1968-12-08	EmpID1.pic	Education includes a BA in psychology from Colorado State University. She also completed (The Art of the Cold Call). Nancy is a member of 'Toastmasters International'.

- **Syntax:**

```
SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders
FROM (Orders
INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID)
GROUP BY LastName
HAVING COUNT(Orders.OrderID) > 5;
```

Operators and clauses: EXISTS operator

- The **EXISTS** operator is used to test for the existence of any record in a subquery.
- It returns TRUE if the subquery returns one or more records.

Syntax:

```
SELECT <column_name(s)>
FROM <table_name>
WHERE EXISTS
(SELECT <column_name>
    FROM <table_name>
    WHERE <condition>);
```

EXISTS operator : Example

- List the suppliers with a product price equal to 20:

SupplierID	SupplierName	ContactName	Address	City	PostalCode	Country	Phone
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	Londona	EC1 4SD	UK	(171) 555-2222
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans	70117	USA	(100) 555-4822

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19

- **Syntax:**

```
SELECT SupplierName FROM Suppliers
WHERE EXISTS
(SELECT ProductName FROM Products
WHERE Products.SupplierID = Suppliers.supplierID AND Price = 20);
```

Operators and clauses: ANY operator

- The **ANY** operator is used to perform a comparison between a single column value and a range of other values.
  - The range of values comes from the execution of a subquery.
  - The ANY operator should be preceded by a column name and a comparison operator like **=, <>, !=, >, >=, <, or <=**.
  - The ANY operator returns a **boolean** value following a comparison operation.
  - It returns a TRUE value if ANY subquery values meet the given condition.

Syntax:

```
SELECT <column_name(s)>
 FROM <table_name>
 WHERE <column_name><comparison operator> ANY
   (SELECT <column_name>
    FROM <table_name>
    WHERE <condition>);
```

ANY operator : Example

- List the ProductName if it finds ANY records in the OrderDetails table has Quantity larger than 100:

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10

Syntax:

```
SELECT ProductName FROM Products
WHERE ProductID = ANY
(SELECT ProductID FROM OrderDetails
WHERE Quantity > 100);
```

Operators and clauses: ALL operator

- The **ALL** operator is used for the same purpose as the ANY operator.
- However, the way it works is a little bit different.
  - It returns a **boolean** value as a result of performing a comparison operation.
  - It returns **TRUE** only if ALL subquery values meet the given condition.
  - It is used with **SELECT, WHERE and HAVING** statements

- **Syntax 1:**

```
SELECT ALL <column_name(s)>
FROM <table_name>
WHERE <condition>);
```

- **Syntax 2:**

```
SELECT <column_name(s)>
FROM <table_name>
WHERE <column_name> ALL
  (SELECT <column_name>
   FROM <table_name>
   WHERE <condition>);
```

ALL operator : Example

❖ List the ProductName if ALL the records in the OrderDetails table has Quantity equal to 10.

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10

❖ **Syntax:**

```
SELECT ProductName FROM Products
WHERE ProductID = ALL
(SELECT ProductID FROM OrderDetails
WHERE Quantity = 10);
```

MySQL Alias: Knowledge check

❖ Which of the following are benefits of using an alias? (Check all that apply)

❑ Combine an output into one column instead of two using a concatenation function.

❑ Create distinct table names when working with multiple tables.

❑ Rename a table or column whose original name is too long or technical.

JOIN Tables: Knowledge check

- You need to query data from two tables. These records should have matching values in both tables, held within a common column. What type of JOIN do you need to create to extract the data?

- ❑ RIGHT JOIN
- ❑ LEFT JOIN
- ❑ INNER JOIN

GROUP BY : Knowledge check

- Which of the following rules must you follow when using GROUP BY syntax?

- ❑ An aggregate function must be followed by name of the column to be queried in parentheses.
- ❑ Place the GROUP BY clause before the WHERE clause.
- ❑ Ensure that the GROUP BY columns are also present in the SELECT command.
- ❑ Begin the statement with a SELECT clause.

Plan of the chapter



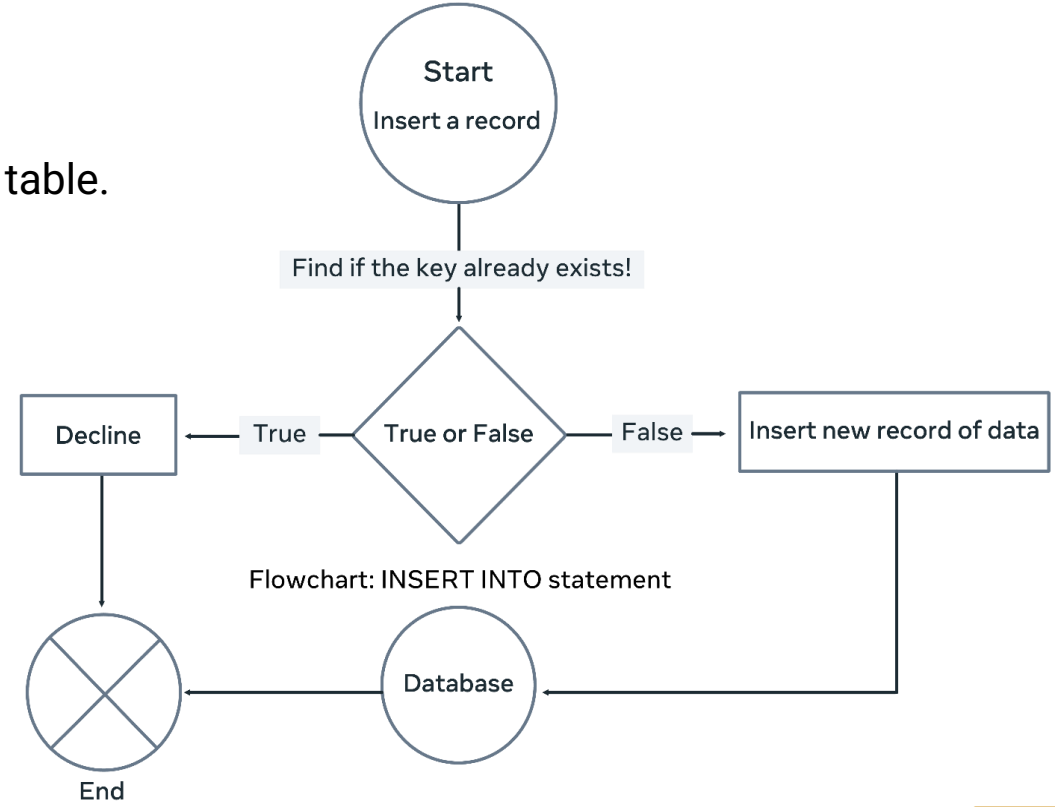
1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

Reminder: How the INSERT INTO statement works

- **INSERT Command:**
  - To add records of data into an existing table.

- **Syntax :**

```
INSERT INTO <table name>
<field name list>)
VALUES (<value list>)
```



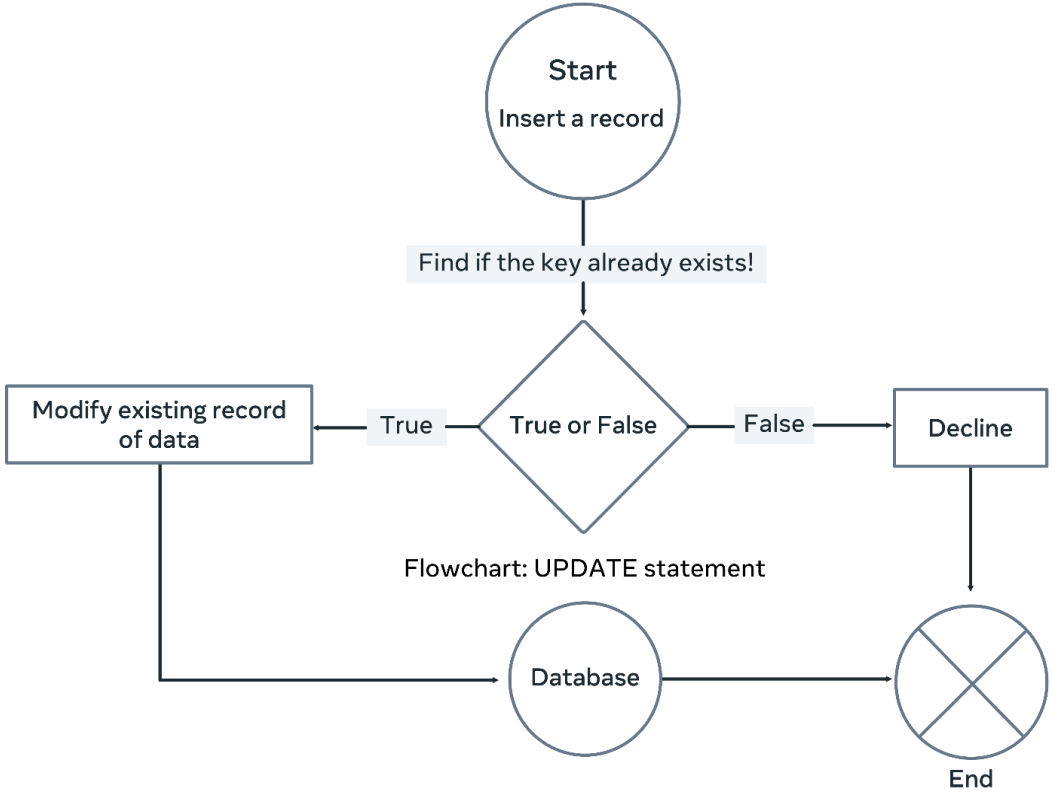
Flowchart: INSERT INTO statement

Reminder: How the UPDATE statement works

- **UPDATE Command:**
  - To modify or update data contained within a table in database.

- **Syntax:**

```
UPDATE <table_name>
SET <field_name> = <new_value>
WHERE <condition>
```



Flowchart: UPDATE statement

MySQL REPLACE INTO statement

- The MySQL **REPLACE** statement is an alternative way to insert and update data in a database table.
- You use it for slightly different purposes than the standard INSERT INTO and UPDATE.
- It is helpful when you want to delete and replace existing records with new ones.
- You can use two types of syntax to insert or update data with the REPLACE statement:

- **First syntax: Like INSERT INTO statement**

```
REPLACE INTO <table_name> (<column_name(1)>, <column_name(2)>, ...)
VALUES (<value1>, <value2>, ...);
```

- **Second syntax: Like UPDATE statement**

```
REPLACE INTO <table_name>
SET <column_name(1)> = <value1>, <column_name(2)>  = <value2>,...;
```

Reminder: How the REPLACE INTO statement works

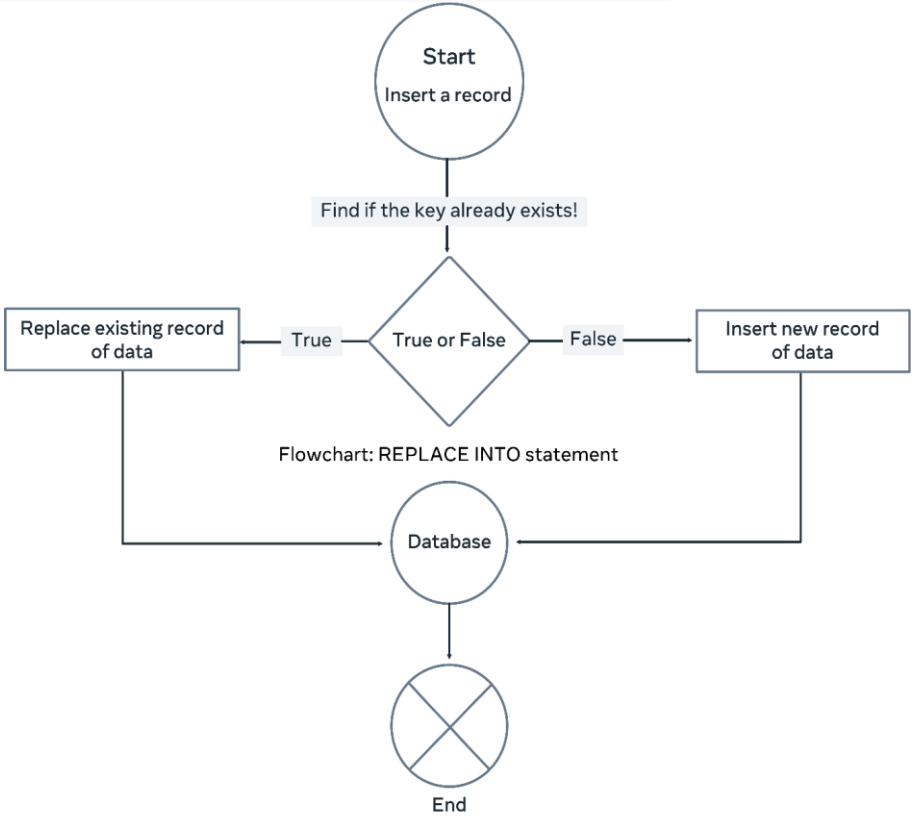
- **REPLACE INTO Command:**

❖ Once the REPLACE INTO statement is used to insert or modify data:

- It determines first whether the new data record already exists in the table.
- It checks if the PRIMARY or the UNIQUE KEY matches one of the existing records.

➢ If there is no matching key, the REPLACE works like a normal INSERT statement.

➢ Otherwise, it deletes the existing record and replaces it with the new one (A sort of modification or update of an existing record).



Flowchart: REPLACE INTO statement

MySQL Constraints

- **Constraints** in MySQL databases are used to define **the rules** that maintain data in tables to ensure ***data validity, accuracy, consistency and reliability***.

- Applying constraints on data in a relational database **controls the type of data** stored in a table.

➢ This will force MySQL to decline processes that interrupt the specified rules.

➢ If there is any conflict between the constraints and the data insertion or update process, the process violating the constraint rule gets aborted.

MySQL Constraints

- **Examples:**
- You must be 18 years old in many countries to apply for a driving license.

➢ You can apply the **SQL CHECK constraints** on the relevant column in the related table in the database to restrict registering any person under 18.

- In most universities nowadays, each student will be registered with an email address containing the university domain name.

➢ You can apply the **UNIQUE constraint** to ensure that the university does not provide two students with the same email address.

MySQL Constraints types

There are three main types of constraints that can be applied in MySQL:

- **Key Constraints**
- **Domain Constraints**
- **Referential Integrity Constraints**

MySQL Constraints types: Key Constraints

- In relational databases, each table must have a primary key that maintains table integrity.
- The **primary key** ensures no duplications of records in the same table.
- It allows identifying each data record using the **primary key value**.
- It must be **unique** in each row of the table, and it is not allowed to contain null values.
- **Example:**

Each citizen living in Tunisia must have a unique personal number, which can be used to access different types of country services.

MySQL Constraints types: Domain Constraints

- **Domain constraints** refer to special rules defined for the values that can be stored for a certain column.
- To apply this, you must specify what data values are allowed and which ones should be rejected.
- **Example:**

You can define a valid range number for users to rate a streaming service that offers a wide variety of TV shows and movies.

This range could be a number between 3 and 10, in which case the user will not be able to insert a value that is more than 10 and less than 3.

MySQL Constraints types: Referential Integrity Constraints

In a relational database, tables are connected via a **foreign key** in one table linked to a **primary key** (or a unique key) in another table.

- The value of the foreign key column in the '**referencing**' table must also exist in the referenced table.
- You will end up with a problem as the **"connection"** between the records of the tables.
- Maintaining referential integrity requires that a foreign key value must have a matching primary key value to link the records of different related tables.

MySQL ALTER TABLE statement

- If the table you initially created in the database needs adjusting.

➢ Use the **ALTER TABLE** statement to:

- to change an existing table structure.
- to add columns,
- to delete columns,
- to change column type,
- to change column constraints,
- to rename columns and the table itself.

Commands to use with the ALTER Statement

- **ADD:** To add a column to an existing table, you can use the **ADD** command followed by the column name and data type. T
- **Syntax:**

```
ALTER TABLE <tablename> ADD <column_name> datatype;
```

- **Example:**

```
ALTER TABLE Employees ADD Email VARCHAR(255);
```

Commands to use with the ALTER Statement

- **MODIFY:** To modify a column, you can use the **MODIFY** command followed by the column that you want to modify.

- **Syntax:**

```
ALTER TABLE <tablename> MODIFY COLUMN <column_name>
datatype;
```

- **Example:**

```
ALTER TABLE Employees MODIFY COLUMN Email VARCHAR(255);
```

Commands to use with the ALTER Statement

- **Adding a FOREIGN KEY to an existing table using ADD:** To create a link between one table and another table, you can use the **FOREIGN KEY** and **REFERENCES** commands along with the **ADD** command.

- **Syntax:**

```
ALTER TABLE <tablename>
ADD FOREIGN KEY <primary_key_column_name>
REFERENCES <link_table_name>(<reference_column_name>);
```

- **Example:**

```
ALTER TABLE Orders ADD FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID);
```

Commands to use with the ALTER Statement

- **DROP COLUMN:** To remove/delete an existing column from a table, you can use the DROP COLUMN command followed by the column that you want to modify.

- **Syntax:**

```
ALTER TABLE <tablename> DROP COLUMN <column_name>
```

- **Example:**

```
ALTER TABLE Employees DROP COLUMN Email;
```

Commands to use with the ALTER Statement

- **CHANGE:** You can use the **CHANGE** command with the ALTER statement to rename a column.

- **Syntax:**

```
ALTER TABLE  <tablename> CHANGE <from_column> <to_column> datatype;
```

- **Example:**

```
ALTER TABLE Employees CHANGE Email BusinessEmail VARCHAR(50);
```

Commands to use with the ALTER Statement

- **RENAME:** To change a table name, the **RENAME** command is used followed by the new name that needs to be given to the table.

- **Syntax:**

```
ALTER TABLE  <tablename> RENAME <new_table_name>;
```

- **Example:**

```
ALTER TABLE OrderStatus RENAME OrderDeliveryStatus;
```

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

©Dr. Kalthoum Zaouali 2026-2027

What is a subquery?

- A **subquery** is **a query within another query =** an **inner query.**
- In a subquery, a SQL SELECT query is placed as a part of another query called an **outer query**.

*! Think of the **inner query as the child query** and the **outer query as the parent query**.*

- Place a subquery in the **SELECT, FROM, or WHERE** clause (it is used in the WHERE most of the time.)
- It always place a subquery within a pair of parentheses.
- There can be subqueries that return a single value, a single row, a single column, or multiple rows of one or more columns.
- It is possible to use operators in a subquery, such as **ALL, ANY** and **SOME** operators followed by a comparison operator like **=, >, >=, <, <=, !=** (or <>).

Types of Subqueries

- **Single row subquery :** Returns zero or one row.
- **Multiple row subquery :** Returns one or more rows.
- **Multiple column subqueries** : Returns one or more columns.
- **Correlated subqueries :** Reference one or more columns in the outer SQL statement. The subquery is known as a correlated subquery because the subquery is related to the outer SQL statement.
- **Nested subqueries :** Subqueries are placed within another subquery.
- **You can use a subquery inside FROM clause, INSERT, UPDATE and DELETE statements.**

Subqueries in the FROM clause

- When you use a subquery in the FROM clause, ***the result returned from a subquery is used as a temporary table*.**
  - This table is referred to as a derived table as well.

- **Syntax:**

```
SELECT some_column, some_column FROM (Subquery) AS alias;
```

Subqueries in the FROM clause: Example

- The following **subquery finds the maximum, minimum, and average** number of items in sale orders:

- **Syntax:**

```
SELECT MAX(items), MIN(items), FLOOR(AVG(items))
FROM (SELECT orderNumber, COUNT(orderNumber) AS items
      FROM orderdetails
      GROUP BY orderNumber)
AS lineitems;
```

Subqueries in INSERT statements

- The SELECT statements can be used in an INSERT statement, as a subquery, when you want to ***retrieve some data and insert them into another table***.

- **Syntax:**

```
INSERT INTO <table1> SELECT <column1,column2,column3> FROM
<table2> WHERE <some_column> <some_operator>(Subquery);
```

Subqueries in INSERT statements: Example

- To insert records into 'agent1' table from 'agents' table with the following conditions:
  - 'agent_code' of agents table must be any 'agent_code' from 'customer' table which satisfies the condition bellow :
    - 'agent_code' of customer table must be any 'agent_code' from 'orders' table which satisfies the condition bellow :
      - same 'agent_code' of 'customer' table should come in a group,
      - 'advance_amount' of 'orders' table must be more than 600,

- **Syntax:**

```
INSERT INTO agent1 SELECT * FROM  agents WHERE agent_code=ANY( SELECT
agent_code FROM customer WHERE agent_code =ANY( SELECT agent_code FROM
orders WHERE  advance_amount>600)GROUP BY agent_code);
```

Subqueries in UPDATE statements

- In the UPDATE statement's WHERE clause, you can use a subquery ***to filter out the records you want to update***.

- **Syntax:**

```
UPDATE <table1> SET <column1> = <some_value>
WHERE <some_column> <some_operator>(Subquery);
```

Subqueries in UPDATE statements: Example

❖ To update the 'agent1' table with following conditions:

- modified value for 'commission' is 'commission'+.02,
- the number 2 is greater than or equal to the number of 'cust_code' from 'customer' table which satisfies the condition:
  - 'agent_code' of 'customer' table and 'agent1' table should match,

❖ **Syntax:**

```
UPDATE agent1 SET commission=commission+.02
WHERE 2>=(SELECT COUNT(cust_code) FROM customer WHERE
customer.agent_code=agent1.agent_code);
```

Subqueries in DELETE statements

- In the WHERE clause of the DELETE statement, you can use a subquery **to filter out the records you want to delete**.

- **Syntax:**

```
DELETE FROM <table1> WHERE <some_column> <some_operator>(Subquery);
```

Subqueries in DELETE statements: Example

- To remove rows from the table 'customer1' with following conditions:
  - 'agent_code' should be any 'agent_code' from 'agents' table which satisfies the condition bellow:
    - 'working_area' of 'agents' table must be ‘Tunisia',

- **Syntax:**

```
DELETE FROM customer1 WHERE agent_code=ANY( SELECT agent_code FROM
agents WHERE working_area='Tunisia');
```

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

What is a VIEW in SQL?

- In SQL, a **VIEW** acts as a **virtual table** that utilizes data stored in existing tables in the database.
  - The virtual table does not store any data itself.
  - Instead, it acts as an interface that provides access to existing data.

- There are several **benefits** to using virtual tables instead of base tables:
  - simplifying access to data,
  - manipulating data,
  - providing security.

Why do you need to use virtual tables?

❖ **Example:**

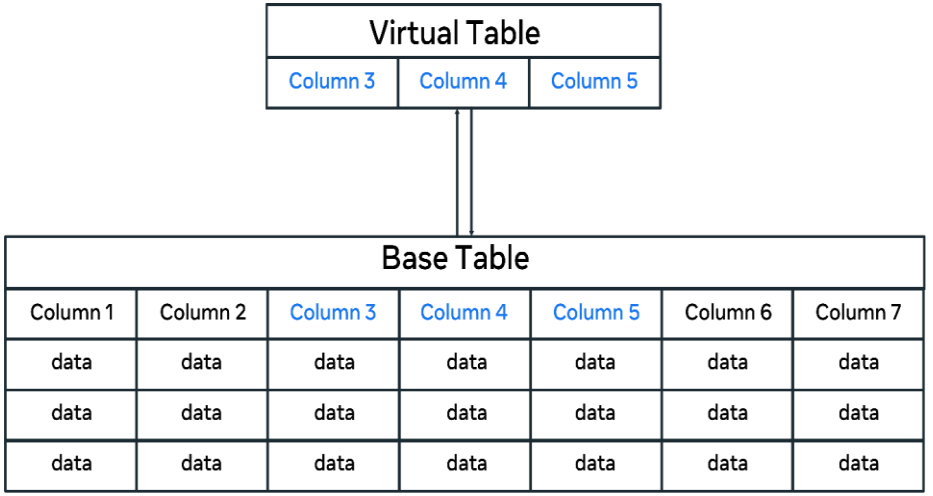
- You have a database with a **base table** with 7 columns named:

column 1, column 2, column 3, column 4, column 5, column 6 and column 7.

- However, you are only interested in viewing and analysing information in columns 3, column 4 and column 5.

➢ In this case, you can create a **virtual table** that contains the three required columns.

➢ This virtual table utilizes the data that exists in these corresponding columns.



➢ *All data in the base table could be treated as part of the virtual table.*

➢ *Though physically, they are stored in the base table, not the virtual table.*

Benefits of using Views

❖ Simplifying access to data:

- You can use virtual tables to simplify how users query and access data in the database.

❖ Example:

- *Lucky Shrub* keeps information in their database about *clients, staff, products* and *orders*.
- Lucky Shrub managers want to display the clients' ID, name and location from the Clients table and the orders they placed, including Order ID, Quantity and Cost from the Orders table.
- ***The managers want to display all of this data in one view without typing complicated queries every time they access this data.***

➢ *To help the managers*, you can create a **virtual table** from the two tables, the clients and the orders: You can combine the client's ID, name and location from the Clients table with the Order ID, Quantity and Cost from the Orders table.

Benefits of using Views

❖ Manipulating data in the base table

- The virtual table allows you to manipulate, filter, and even update data in the base table if necessary.

❖ Example:

- The Orders table shows information about the Order ID, Client ID, Product ID, Quantity and Cost:

➢ The following virtual table, **"OrdersView,"** has been created from the Orders table using only three required columns:

```
CREATE VIEW OrdersView AS SELECT OrderID,
Quantity, Cost FROM Orders;
```

```
mysql> select * from Orders;
+---------+----------+-----------+----------+---------+
| OrderID | ClientID | ProductID | Quantity | Cost    |
+---------+----------+-----------+----------+---------+
|       1 | Cl1      | P1        |       10 |  500.00 |
|       2 | Cl2      | P2        |        5 |  200.00 |
|       3 | Cl3      | P3        |       20 |  800.00 |
|       4 | Cl4      | P4        |       15 |  150.00 |
|       5 | Cl3      | P3        |       10 |  450.00 |
|       6 | Cl2      | P2        |        5 |  800.00 |
|       7 | Cl1      | P4        |       22 | 1200.00 |
|       8 | Cl1      | P1        |       15 |  150.00 |
+---------+----------+-----------+----------+---------+
```

```
mysql> SELECT * FROM OrdersView;
+---------+----------+---------+
| OrderID | Quantity | Cost    |
+---------+----------+---------+
|       1 |       10 |  500.00 |
|       2 |        5 |  200.00 |
|       3 |       20 |  800.00 |
|       4 |       15 |  150.00 |
|       5 |       10 |  450.00 |
|       6 |        5 |  800.00 |
|       7 |       22 | 1200.00 |
|       8 |       15 |  150.00 |
+---------+----------+---------+
```

Benefits of using Views

❖ **Example:**

- Lucky Shrub need to change the cost of the second record (order id = 2) to 250.

➢ You can make this change directly in the virtual table using the UPDATE statement:

```
UPDATE OrdersView
SET Cost = 250
WHERE OrderID = 2;
```

```
mysql> SELECT * FROM Orders;
+---------+----------+-----------+----------+---------+
| OrderID | ClientID | ProductID | Quantity | Cost    |
+---------+----------+-----------+----------+---------+
|       1 | Cl1      | P1        |       10 |  500.00 |
|       2 | Cl2      | P2        |        5 |  250.00 |
|       3 | Cl3      | P3        |       20 |  800.00 |
|       4 | Cl4      | P4        |       15 |  150.00 |
|       5 | Cl3      | P3        |       10 |  450.00 |
|       6 | Cl2      | P2        |        5 |  800.00 |
|       7 | Cl1      | P4        |       22 | 1200.00 |
|       8 | Cl1      | P1        |       15 |  150.00 |
+---------+----------+-----------+----------+---------+
```

Benefits of using Views

❖ Virtual tables support database security:

- When you design virtual tables, you create interfaces for users to access relevant data in your database, similarly to the base tables.
- However, you can only show the required data and hide what is not needed.

❖ Example:

- You created a virtual table called "OrdersView" from the Orders table with information about the orders ID, quantity and cost only.
- ***This restricts the users from viewing specific columns***: They only view what you want them to view.

➢ Users will not be able to read information about client and product IDs.

➢ This may be sensitive information you need to hide from other database users.

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

©Dr. Kalthoum Zaouali 2026-2027

What is MySQL Function?

- A function is a code that performs an operation and returns a result.
- Functions are used to manipulate data in a database table.
- MySQL has many built-in functions that fall under different categories. These include:
  - String manipulation functions
  - Date and time functions
  - Numeric functions
  - Comparison functions
  - And control flow functions

Numeric functions: CEIL

Function	Description
ABS	Returns the absolute value of a number
ACOS	Returns the arc cosine of a number
ASIN	Returns the arc sine of a number
ATAN	Returns the arc tangent of one or two numbers
ATAN2	Returns the arc tangent of two numbers
AVG	Returns the average value of an expression
CEIL	Returns the smallest integer value that is >= to a number
CEILING	Returns the smallest integer value that is >= to a number
COS	Returns the cosine of a number
COT	Returns the cotangent of a number
COUNT	Returns the number of records returned by a select query
DEGREES	Converts a value in radians to degrees

©Dr. Kalthoum Zaouali 2026-2027

Numeric functions: CEIL

Function	Description
DIV	Used for integer division
EXP	Returns e raised to the power of a specified number
FLOOR	Returns the largest integer value that is <= to a number
GREATEST	Returns the greatest value of the list of arguments
LEAST	Returns the smallest value of the list of arguments
LN	Returns the natural logarithm of a number
LOG	Returns the natural logarithm of a number, or the logarithm of a number to a specified base
LOG10	Returns the natural logarithm of a number to base 10

Numeric functions: CEIL

Function	Description
LOG2	Returns the natural logarithm of a number to base 2
MAX	Returns the maximum value in a set of values
MIN	Returns the minimum value in a set of values
MOD	Returns the remainder of a number divided by another number
PI	Returns the value of PI
POW	Returns the value of a number raised to the power of another number
POWER	Returns the value of a number raised to the power of another number
RADIANS	Converts a degree value into radians
RAND	Returns a random number

Numeric functions: CEIL

Function	Description
ROUND	Rounds a number to a specified number of decimal places
SIGN	Returns the sign of a number
SIN	Returns the sine of a number
SQRT	Returns the square root of a number
SUM	Calculates the sum of a set of values
TAN	Returns the tangent of a number
TRUNCATE	Truncates a number to the specified number of decimal places

Numeric functions: FLOOR

- The FLOOR function does the opposite of CEIL.
- It returns the largest integer value not greater than the passed value.
- It returns NULL if the passed value is NULL.

- **Syntax:** SELECT FLOOR(VALUE);
- **Examples**:

1. Passing a value of 1.23 to the FLOOR function: SELECT FLOOR(1.23);

- It returns a value of 1.

2. Passing a value of 15.50 to the FLOOR function: SELECT FLOOR(15.50);

- It returns 15 as it is the smallest integer value that is greater than or equal to 15.50.

String functions

Function	Description
ASCII	Returns the ASCII value for the specific character
CHAR_LENGTH	Returns the length of a string (in characters)
CHARACTER_LENGTH	Returns the length of a string (in characters)
CONCAT	Adds two or more expressions together
CONCAT_WS	Adds two or more expressions together with a separator
FIELD	Returns the index position of a value in a list of values
FIND_IN_SET	Returns the position of a string within a list of strings
FORMAT	Formats a number to a format like "#,###,###.##", rounded to a specified number of decimal places
INSERT	Inserts a string within a string at the specified position and for a certain number of characters
INSTR	Returns the position of the first occurrence of a string in another string

String functions

Function	Description
LCASE	Converts a string to lower-case
LEFT	Extracts a number of characters from a string (starting from left)
LENGTH	Returns the length of a string (in bytes)
LOCATE	Returns the position of the first occurrence of a substring in a string
LOWER	Converts a string to lower-case
LPAD	Left-pads a string with another string, to a certain length
LTRIM	Removes leading spaces from a string
MID	Extracts a substring from a string (starting at any position)
POSITION	Returns the position of the first occurrence of a substring in a string
REPEAT	Repeats a string as many times as specified
UCASE	Converts a string to upper-case
UPPER	Converts a string to upper-case

String functions : FORMAT

- The **FORMAT** function formats the number passed into a format like '#,###,###.##', rounded to the specified number of decimal places.
- It returns the result as a string.
- **Syntax:**

```
SELECT FORMAT(number_to_be_formatted, number_of_decimal_places);
```

- **Example:** `SELECT FORMAT(3750.753, 2);`
- It returns 3,750.75 as it formats the number (3750.753, 2) as "#,###.##" and rounds it within two decimal places.
- If the number of decimal places is 0, the result has no decimal point or fractional part.
- If the number to be formatted or the number of decimal places is NULL, then the function returns NULL.

Date functions

Function	Description
ADDDATE	Adds a time/date interval to a date and then returns the date
ADDTIME	Adds a time interval to a time/datetime and then returns the time/datetime
CURDATE	Returns the current date
CURRENT_DATE	Returns the current date
CURRENT_TIME	Returns the current time
CURRENT_TIMESTAMP	Returns the current date and time
CURTIME	Returns the current time
DATE	Extracts the date part from a datetime expression
DATEDIFF	Returns the number of days between two date values

Date functions

Function	Description
DATE_ADD	Adds a time/date interval to a date and then returns the date
DATE_FORMAT	Formats a date
DATE_SUB	Subtracts a time/date interval from a date and then returns the date
DAY	Returns the day of the month for a given date
DAYNAME	Returns the weekday name for a given date
DAYOFMONTH	Returns the day of the month for a given date
DAYOFWEEK	Returns the weekday index for a given date
DAYOFYEAR	Returns the day of the year for a given date
EXTRACT	Extracts a part from a given date
FROM_DAYS	Returns a date from a numeric datevalue

Date functions

Function	Description
HOUR	Returns the hour part for a given date
LAST_DAY	Extracts the last day of the month for a given date
LOCALTIME	Returns the current date and time
LOCALTIMESTAMP	Returns the current date and time
MAKEDATE	Creates and returns a date based on a year and a number of days value
MAKETIME	Creates and returns a time based on an hour, minute, and second value
MICROSECOND	Returns the microsecond part of a time/datetime
MINUTE	Returns the minute part of a time/datetime

Date functions

Function	Description
MONTH	Returns the month part for a given date
MONTHNAME	Returns the name of the month for a given date
NOW	Returns the current date and time
PERIOD_ADD	Adds a specified number of months to a period
PERIOD_DIFF	Returns the difference between two periods
QUARTER	Returns the quarter of the year for a given date value
SECOND	Returns the seconds part of a time/datetime
SEC_TO_TIME	Returns a time value based on the specified seconds
STR_TO_DATE	Returns a date based on a string and a format

Date functions

Function	Description
SUBDATE	Subtracts a time/date interval from a date and then returns the date
SUBTIME	Subtracts a time interval from a datetime and then returns the time/datetime
SYSDATE	Returns the current date and time
TIME	Extracts the time part from a given time/datetime
TIME_FORMAT	Formats a time by a specified format
TIME_TO_SEC	Converts a time value into seconds
TIMEDIFF	Returns the difference between two time/datetime expressions
TIMESTAMP	Returns a datetime value based on a date or datetime value

Date functions

Function	Description
TO_DAYS	Returns the number of days between a date and date "0000-00-00"
WEEK	Returns the week number for a given date
WEEKDAY	Returns the weekday number for a given date
WEEKOFYEAR	Returns the week number for a given date
YEAR	Returns the year part for a given date
YEARWEEK	Returns the year and week number for a given date

Date functions: ADDDATE

- The **ADDDATE** function is used to perform arithmetic with dates. It comes in two forms:
- **Syntax 1:**

```
SELECT ADDDATE(date, INTERVAL expr unit)
```

- The first argument specifies the starting date or datetime value in the first form.
- The second argument is an expression that determines the interval value to be added to the starting date. It has three parts: INTERVAL is a keyword, expr represents a quantity, and unit represents the unit for interpreting the quantity; such as HOUR, DAY, or WEEK

- **Example:**

```
SELECT ADDDATE("2020-05-10", INTERVAL 5 DAY)
```

- The following code returns 2020-05-15 because it adds 5 days to the specified date.

Date functions: ADDDATE

❖ **Syntax 2 :**

```
SELECT ADDDATE(date, days);
```

➢ In the second form, the first argument is the same and the second argument is the integer, or number of days, to be added to the given date in the first argument.

❖ **Example:**

```
SELECT ADDDATE("2020-06-15", 10);
```

➢ The following code returns 2020-06-25 because it adds 5 days to the specified date.

Date functions: QUARTER

- The **QUARTER** function is also a very versatile function that returns the quarter of the year for the given date. The return value is in the range 1 to 4, or NULL if date is NULL.
- **Syntax :** `SELECT QUARTER(date_value);`
- **Example:** `SELECT QUARTER("2020-09-15");`
- The following code returns a value of 3.
- The QUARTER() function returns the quarter of the year for the given date value.

- The **DATE, MONTH, YEAR and DAY** functions extract the date, month, year and 'day of the month' parts respectively, of a given date or date time expression.

Advanced functions

Function	Description
BIN	Returns a binary representation of a number
BINARY	Converts a value to a binary string
CASE	Goes through conditions and return a value when the first condition is met
CAST	Converts a value (of any type) into a specified datatype
COALESCE	Returns the first non-null value in a list
CONNECTION_ID	Returns the unique connection ID for the current connection
CONV	Converts a number from one numeric base system to another
CONVERT	Converts a value into the specified datatype or character set
CURRENT_USER	Returns the user name and host name for the MySQL account that the server used to authenticate the current client

Advanced functions

Function	Description
DATABASE	Returns the name of the current database
IF	Returns a value if a condition is TRUE, or another value if a condition is FALSE
IFNULL	Return a specified value if the expression is NULL, otherwise return the expression
ISNULL	Returns 1 or 0 depending on whether an expression is NULL
LAST_INSERT_ID	Returns the AUTO_INCREMENT id of the last row that has been inserted or updated in a table
NULLIF	Compares two expressions and returns NULL if they are equal. Otherwise, the first expression is returned
SESSION_USER	Returns the current MySQL user name and host name

Advanced functions

Function	Description
SYSTEM_USER	Returns the current MySQL user name and host name
USER	Returns the current MySQL user name and host name
VERSION	Returns the current version of the MySQL database

Comparison functions: COALESCE

- Like GREATEST, LEAST and ISNULL, **COALESCE** is another comparison function that takes several arguments and returns the first non-NULL argument.
- In case all arguments are NULL, the COALESCE function returns NULL.

- **Syntax :** `SELECT COALESCE (value1, value2);`
- This function is particularly useful when you replace a column value with a NULL value with some other value.

- **Example:** `SELECT COALESCE(NULL, 'Pristini', NULL, 'Database');`
- The following code returns `Pristini`, because the COALESCE() function returns the first non-null value in a list.

Control flow functions: IFNULL

- **IFNULL** is a MySQL function that lets you evaluate conditions and decide how the query should be executed accordingly.
  - It returns the second argument. The two arguments can be literal values or expressions.

- **Syntax:**
```
SELECT IFNULL(evaluated_expression, alternative_value);
```

- **Example:**
```
SELECT IFNULL(NULL, "Coursera");
```
  - The following code returns Coursera as the value because the evaluated expression is NULL.

Control flow functions: NULLIF

- The **NULLIF** function takes in two values or expressions.
- If they're equal then it returns NULL. Otherwise, it returns the first value or expression.

- **Syntax:**
```
SELECT NULLIF(expression1, expression2);
```

- **Example:**
```
SELECT NULLIF (10, 15);
```
- The following NULLIF() function returns 10, as it compares the two expressions (10 and 15) and returns NULL if they are equal. Otherwise, the first expression is returned.

Plan of the chapter



1. MySQL DBMS: Build, modify and query relational databases with SQL
2. MySQL DBMS: Update databases
3. MySQL DBMS: Subqueries
4. MySQL DBMS: Views
5. MySQL DBMS: Functions
6. MySQL DBMS: Stored procedures

What is a stored procedure?

- The stored procedure in MySQL is a set of SQL instructions wrapped within the CREATE PROCEDURE statement to achieve a particular objective.

- **Example:**

```
CREATE PROCEDURE GetItalianCustomers() SELECT * FROM Customers WHERE Country = “Italy”;
```

- This stored procedure returns all customers from Italy by just calling the identifier of the procedure as follows:

```
CALL GetItalianCustomers();
```

Why use stored procedures?

- The main idea behind creating **stored procedures** is to **create reusable code that can be invoked and executed efficiently**.
- Instead of typing the same code repeatedly, you can save your blocks of code in a stored procedure as a prepared query that you can use whenever needed.
- You can create as many procedures as you need.
- Each procedure can have a unique name, **zero or several parameters and contain blocks of code**, including **SQL statements, variables and conditions.**

➢ Stored procedures make the code **more consistent, reusable and easier to use and maintain.**

When to create a stored procedure with parameters?

- Each procedure must achieve a clear objective.

- **Example:** Suppose you expect users to input a value when calling a procedure.
  - You need to specify a parameter with a suitable data type.
  - Otherwise, the procedure will be created with a parameter of zero.

Example

- The Products table from the Lucky Shrub database includes 3 columns:

- You can create a basic stored procedure with empty (or zero) parameters and one single SQL statement by typing the following query:

```
CREATE PROCEDURE GetProductsDetails()

SELECT * FROM Products;
```

```
mysql> select * from Products;
+-----------+----------------------+-------+
| ProductID | Item                 | Price |
+-----------+----------------------+-------+
| P1        | Artificial grass bags | 50.00 |
| P2        | Wood panels          | 20.00 |
| P3        | Patio slates         | 40.00 |
| P4        | Sycamore trees       | 10.00 |
| P5        | Trees and Shrubs     | 50.00 |
| P6        | Water fountain       | 80.00 |
+-----------+----------------------+-------+
```

- This procedure returns all records from the Products table. No argument (user input value) is expected to be passed to the procedure to execute it. That is why it has no parameter within the parentheses.

Example

- To create a stored procedure with one or more parameters to process the query based on a user-specified value.
- For example, you can create a stored procedure to retrieve data from the products table based on the user input of a specific price:

```
CREATE PROCEDURE GetProductsBasedOnPrice
(inputPrice INT)

SELECT * FROM Products WHERE Price <=
inputPrice;
```

```
mysql> select * from Products;
+-----------+----------------------+-------+
| ProductID | Item                 | Price |
+-----------+----------------------+-------+
| P1        | Artificial grass bags | 50.00 |
| P2        | Wood panels          | 20.00 |
| P3        | Patio slates         | 40.00 |
| P4        | Sycamore trees       | 10.00 |
| P5        | Trees and Shrubs     | 50.00 |
| P6        | Water fountain       | 80.00 |
+-----------+----------------------+-------+
```

- In this statement, you declare a parameter with an INT datatype to pass an integer value into the stored procedure to make the query act based on this value.

Example

- You can return all data from the Products table where the product price is less than or equal to the user input value.
- **Syntax:**

```
CALL GetProductsBasedOnPrice (50);
```

- This lists all products with prices less than or equal to $50 as shown on the screen below.

```
mysql> CALL GetLowestPriceProducts(50);
+-----------+-----------------------+-------+
| ProductID | Item                  | Price |
+-----------+-----------------------+-------+
| P1        | Artificial grass bags | 50.00 |
| P2        | Wood panels           | 20.00 |
| P3        | Patio slates          | 40.00 |
| P4        | Sycamore trees        | 10.00 |
| P5        | Trees and Shrubs      | 50.00 |
+-----------+-----------------------+-------+
```

References

[1] Book, Ramakrishnan, Raghu, et al. "DATABASE MANAGEMENT SYSTEMS SOLUTIONS MANUAL THIRD EDITION.“

[2] Medium web site; SQL TUTORIAL COURSE FROM CRAIG DICKSON, Coding and Implementing a Relational Database using MySQL;Coding and Implementing a Relational Database using MySQL | by Craig Dickson | Towards Data Science

[3] DataCamp web site; MySQL Tutorial: A Comprehensive Guide for Beginners; MySQL Tutorial: A Comprehensive Guide for Beginners | DataCamp

[4] W3 School web site; MySQL Tutorial; MySQL Tutorial (w3schools.com)